HDOC Day 41 by Trishil Chauhan

Menu-Driven C Program: Paragraph String Operations

Aim: Read two paragraphs and perform longest/shortest word detection, repeated-space removal, title-case conversion, and case-insensitive word frequency analysis using separate user-defined functions.

Complete C Program

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 2000
#define MAX_WORDS 400

void readParagraph(char paragraph[], size_t size) {
    if (fgets(paragraph, (int)size, stdin) != NULL) {
        paragraph[strcspn(paragraph, "\n")] = '\0';
    }
}

void findLongestAndShortestWords(const char paragraph[],
                                 char longestWord[], char shortestWord[]) {
    char copy[MAX];
    strcpy(copy, paragraph);
    longestWord[0] = shortestWord[0] = '\0';
    char *word = strtok(copy, " \t\r\n.,;:!?\"'()[]{}-");
    while (word != NULL) {
        size_t length = strlen(word);
        if (longestWord[0] == '\0' || length > strlen(longestWord))
            strcpy(longestWord, word);
        if (shortestWord[0] == '\0' || length < strlen(shortestWord))
            strcpy(shortestWord, word);
        word = strtok(NULL, " \t\r\n.,;:!?\"'()[]{}-");
    }
}

void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[]) {
    int i = 0, j = 0, previousSpace = 0;
    while (paragraph[i] != '\0') {
        unsigned char ch = (unsigned char)paragraph[i++];
        if (isspace(ch)) {
            if (!previousSpace) modifiedParagraph[j++] = ' ';
            previousSpace = 1;
        } else {
            modifiedParagraph[j++] = (char)ch;
            previousSpace = 0;
        }
    }
    if (j > 0 && modifiedParagraph[j - 1] == ' ') j--;
    modifiedParagraph[j] = '\0';
}

void convertToTitleCase(const char paragraph[], char titleCaseParagraph[]) {
    int newWord = 1, i = 0;

    while (paragraph[i] != '\0') {
        unsigned char ch = (unsigned char)paragraph[i];
        if (isalpha(ch)) {
            titleCaseParagraph[i] = (char)(newWord ? toupper(ch) : tolower(ch));
            newWord = 0;
        } else {
            titleCaseParagraph[i] = (char)ch;
            if (isspace(ch) || ispunct(ch)) newWord = 1;
        }
        i++;
    }
    titleCaseParagraph[i] = '\0';
}

void findWordFrequencies(const char paragraph[]) {
    char copy[MAX], words[MAX_WORDS][MAX];
```

```
    int counts[MAX_WORDS] = {0}, total = 0;
    strcpy(copy, paragraph);
    char *token = strtok(copy, " \t\r\n.,;:!?\"'()[]{}-");
    while (token != NULL) {
        for (int i = 0; token[i] != '\0'; i++)
            token[i] = (char)tolower((unsigned char)token[i]);
        int found = -1;
        for (int i = 0; i < total; i++) {
            if (strcmp(words[i], token) == 0) { found = i; break; }
        }
        if (found >= 0) counts[found]++;
        else if (total < MAX_WORDS) {
            strcpy(words[total], token);
            counts[total++] = 1;
        }
        token = strtok(NULL, " \t\r\n.,;:!?\"'()[]{}-");
    }
    if (total == 0) { printf("No words found.\n"); return; }
    printf("%-25s %s\n", "Word", "Frequency");
    for (int i = 0; i < total; i++)
        printf("%-25s %d\n", words[i], counts[i]);
}

int main(void) {
    char paragraphs[2][MAX], longest[MAX], shortest[MAX], modified[MAX];
    int choice;
    printf("Enter paragraph 1: ");
    readParagraph(paragraphs[0], MAX);
    printf("Enter paragraph 2: ");
    readParagraph(paragraphs[1], MAX);
    do {
        printf("\n--- PARAGRAPH MENU ---\n");
        printf("1. Read two paragraphs again\n");
        printf("2. Longest and shortest words\n");
        printf("3. Remove repeated spaces\n");
        printf("4. Convert to title case\n");
        printf("5. Word frequencies (case-insensitive)\n");
        printf("6. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) {
            printf("Invalid input.\n");
            break;
        }
        int c;
        while ((c = getchar()) != '\n' && c != EOF) {}
        if (choice == 1) {
            for (int i = 0; i < 2; i++) {
                printf("Enter paragraph %d: ", i + 1);
                readParagraph(paragraphs[i], MAX);
            }
        } else if (choice >= 2 && choice <= 5) {
            for (int i = 0; i < 2; i++) {
                printf("\nParagraph %d: %s\n", i + 1, paragraphs[i]);
                switch (choice) {
                    case 2:
                        findLongestAndShortestWords(paragraphs[i], longest, shortest);
                        if (longest[0] == '\0') printf("No words found.\n");
                        else printf("Longest: %s\nShortest: %s\n", longest, shortest);
                        break;
                    case 3:
                        removeRepeatedSpaces(paragraphs[i], modified);
                        printf("Without repeated spaces: %s\n", modified);
                        break;
                    case 4:
                        convertToTitleCase(paragraphs[i], modified);
                        printf("Title case: %s\n", modified);
                        break;
                    case 5:
                        findWordFrequencies(paragraphs[i]);
                        break;
                }
            }
        } else if (choice == 6) printf("Exiting program.\n");
        else printf("Invalid choice. Try again.\n");
    } while (choice != 6);
    return 0;
}
```

Sample Run

```
Enter paragraph 1: Hello   hello World
Enter paragraph 2: C programming is FUN
Enter choice: 5
Paragraph 1: Hello   hello World
Word                      Frequency
hello                     2
world                     1
Paragraph 2: C programming is FUN
Word                      Frequency
c                         1
programming               1
is                        1
fun                       1
```

Notes

The program uses fgets() to read spaces, strtok() on a copy of the input to preserve the original paragraph, and unsigned-char casts for safe ctype operations. Equal-length ties keep the first encountered word. Whitespace is normalized to a single space; trailing whitespace is removed. A maximum of 400 distinct words is supported per paragraph.